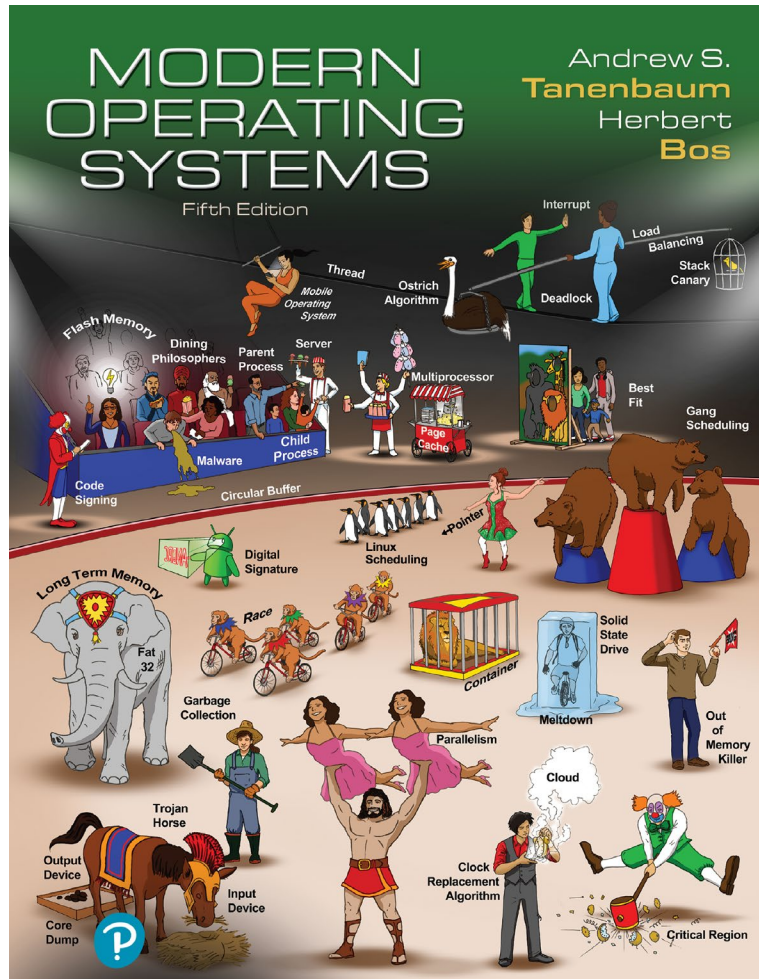


Modern Operating Systems

Fifth Edition



Chapter 2 Processes and Threads Dr. Xiaoyuan Suo

Processes and threads

- Introduction to Processes
 - The Process Model
 - Process Management
 - Process States
 - Threads
 - Signal Handling
- Inter-Process Communication
 - IPC Mechanisms
 - Classical IPC Problems
- Scheduling

The Process Model

Process = Program in execution

- How many processes for each program?
- A fundamental operating system **abstraction**
- Allows the OS to simplify:
 - Resource **allocation**
 - Resource **accounting**
 - Resource **limiting**
- OS maintains information on the resources and the internal state of every single process in the system

The Process Model (1)

- Single program counter
- Each process in unique location
- CPU switches back and forth from process to process

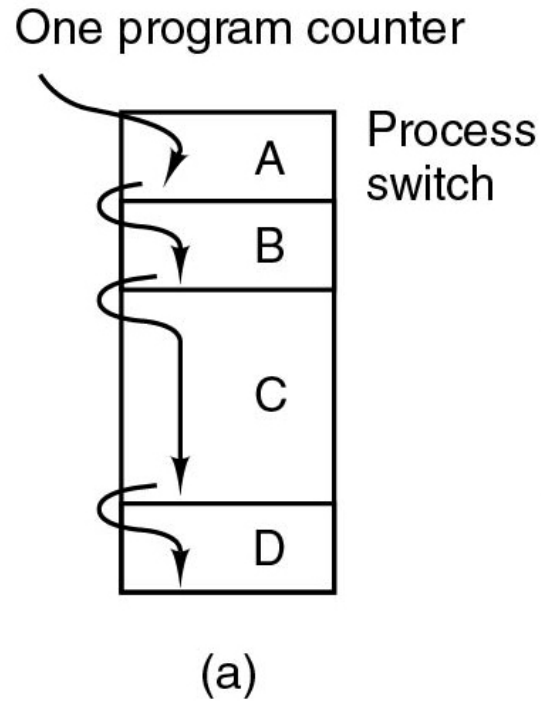
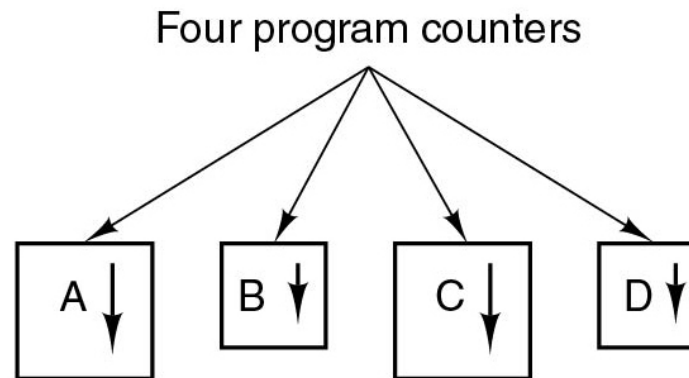


Figure 2.1 (a) Multiprogramming of four programs.

The Process Model (2)

- Each process has own flow of control (own logical program counter)
- Each time we switch processes, we save the program counter of first process and restore the program counter of the second



(b)

Figure 2.1 (b) Conceptual model of four independent, sequential processes.

The Process Model (3)

- All processes make progress, but only one is active at any given time

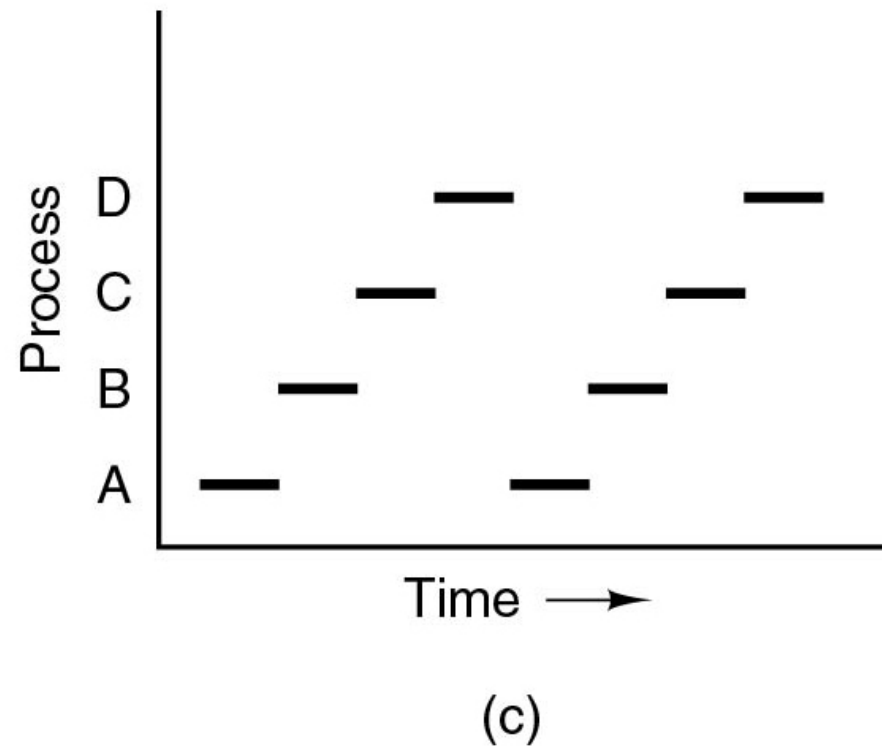
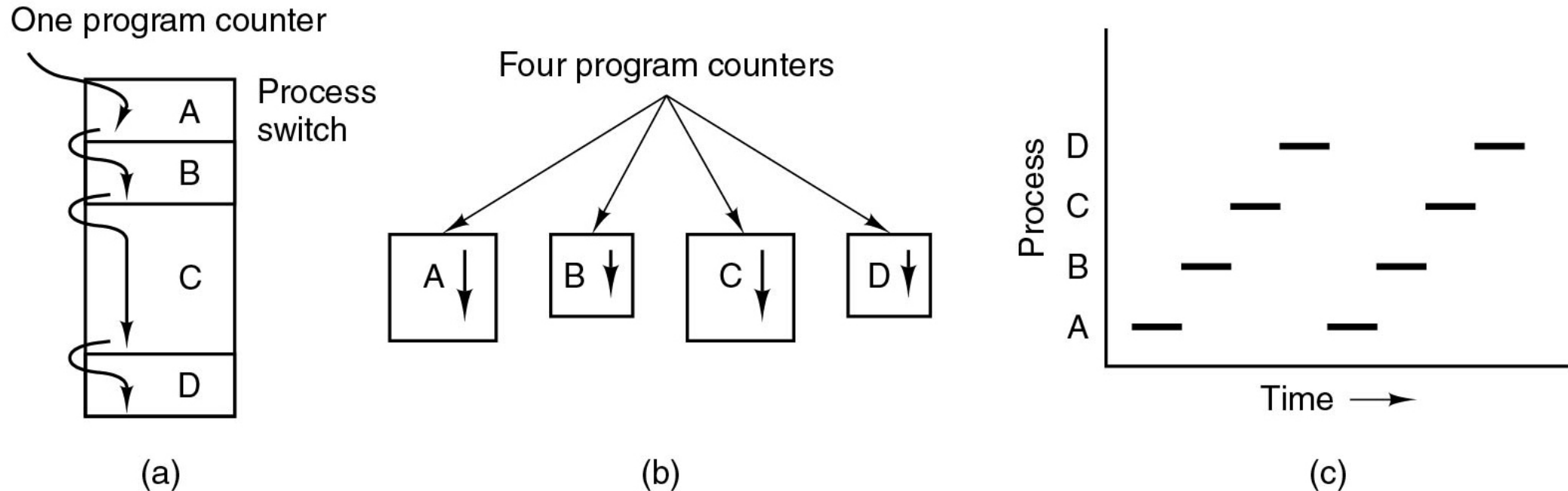


Figure 2.1 (c) Only one program is active at once.

Concurrent Processes



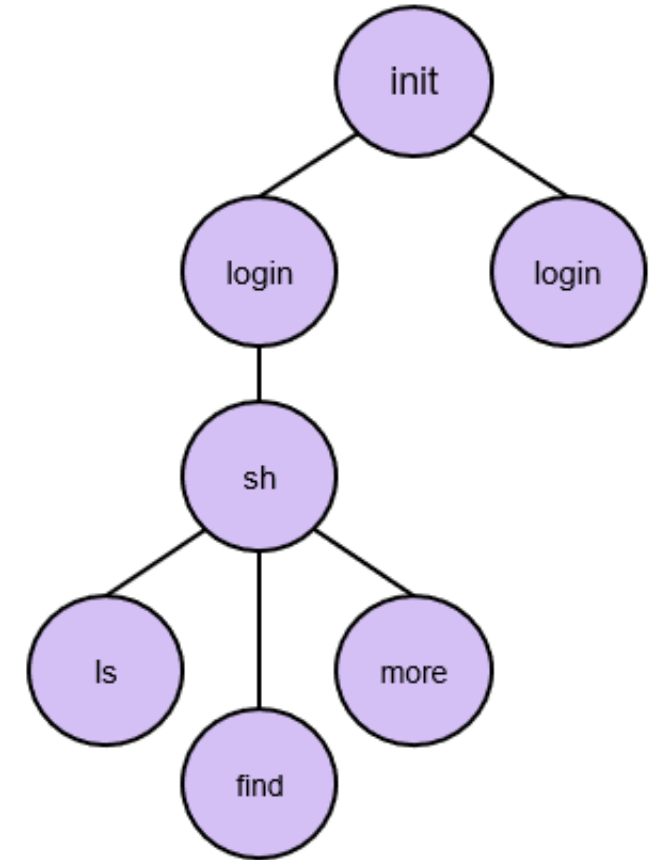
- In principle, multiple processes are mutually independent
- They need explicit means to interact with each other
- The CPU can be allocated in turns to different processes
- OS normally offers **no timing or ordering** guarantees

Process Hierarchies

OS typically creates only 1 `init` process

Subprocesses created independently:

- A parent process can create a child process
- This results in a tree-like structure and process groups



Process Creation

Four principal events that cause processes to be created:

1. System initialization
2. Execution of a process creation system call by a running process
3. A user request to create a new process
4. Initiation of a batch job

Process Termination

Typical conditions which terminate a process:

1. Normal exit (voluntary).
2. Error exit (voluntary).
3. Fatal error (involuntary).
4. Killed by another process (involuntary).

Process Management

- fork: create a new process
 - Child is a “private” **clone** of the parent
 - Shares **some** resources with the parent
- exec: execute a new process image
 - Used in combination with fork
 - exec on Windows?
- exit: cause voluntary process termination
 - Exit status returned to the parent
 - Involuntary process termination?
- kill: send a signal to a process (or group)
 - Can cause involuntary process termination

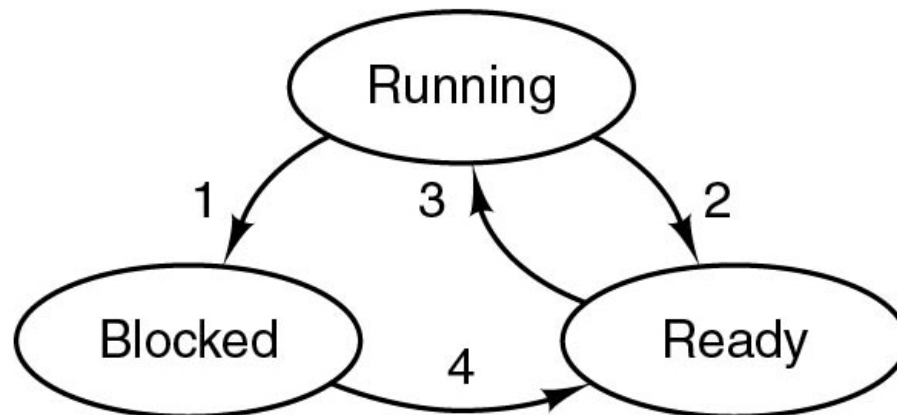
Process States (1)

Three states a process may be in:

1. Running (actually using the CPU at that instant).
2. Ready (runnable; temporarily stopped to let another process run).
3. Blocked (unable to run until some external event happens).

Process States (2)

- The OS allocates resources (e.g., CPU) to processes
- To allocate the CPU, the OS needs to track **process states**:
 - **Running**: the process is currently executed by the CPU
 - **Blocked**: the process is waiting for available resources
 - **Ready**: the process is ready to be selected
- The scheduler (de)allocates the CPU (see transitions 2&3)



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

Figure 2.2 A process can be in running, blocked, or ready state. Transitions between these states are as shown.

Process States (3)

- Scheduler periodically switches processes
- Sequential processes lay on the layer above
- This leads to a simple process organization
- What are the missing fundamental OS abstractions?

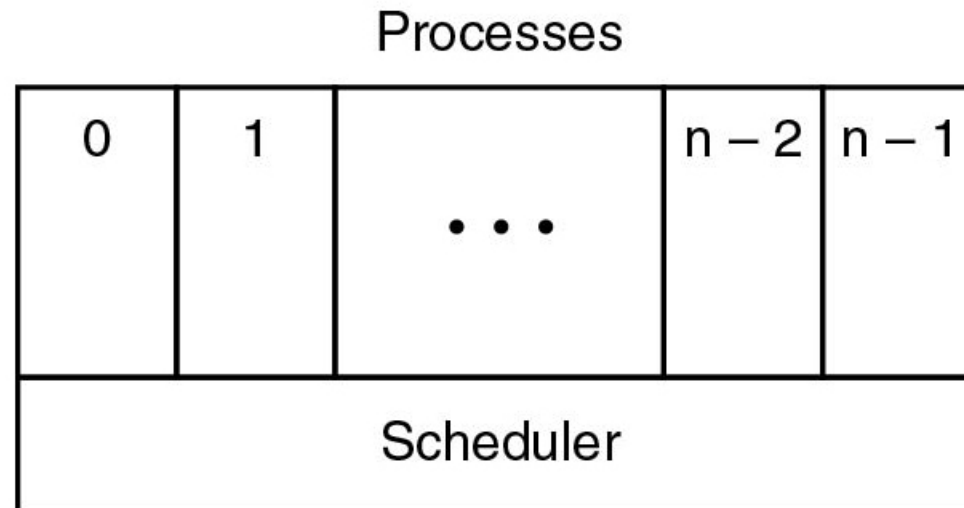


Figure 2.3 The lowest layer of a process-structured operating system handles interrupts and scheduling. Above that layer are sequential processes.

Information Associated with a Process

- ID (PID), User (UID), Group (GID)
- Memory address space
- Hardware registers (e.g., program counter)
- Open files
- Signals
- ...

This information is stored in the operating system's **Process Table**

Process Control Blocks

Process management	Memory management	File management
Registers Program counter Program status word Stack pointer Process state Priority Scheduling parameters Process ID Parent process Process group Signals Time when process started CPU time used Children's CPU time Time of next alarm	Pointer to text segment info Pointer to data segment info Pointer to stack segment info	Root directory Working directory File descriptors User ID Group ID

Figure 2.4 Some of the fields of a typical process-table entry

Interrupts

- Idea: to deallocate the CPU in favor of the scheduler, we rely on hardware-provided interrupt handling support
- Allows the scheduler to periodically get control, i.e., whenever the hardware generates an interrupt
- Interrupt vector:
 - Associated with each I/O device and interrupt line
 - Part of the interrupt descriptor table (IDT)
 - Contains the start address of an OS-provided internal procedure (interrupt handler)
- The interrupt handler continues the execution
- Interrupt types: sw, hw device (async), exceptions

Implementation of Processes (2 of 2)

Skeleton of what the lowest level of the operating system does when an interrupt occurs.

1. Hardware stacks program counter, etc.
 2. Hardware loads new program counter from interrupt vector.
 3. Assembly language procedure saves registers.
 4. Assembly language procedure sets up new stack.
 5. C interrupt service runs (typically reads and buffers input).
 6. Scheduler decides which process is to run next.
 7. C procedure returns to the assembly code.
 8. Assembly language procedure starts up new current process.
- Every time an interrupt occurs, the scheduler gets control → acts as a mediator
 - A process cannot give the CPU to another process (context switch) without going through the scheduler

Modeling Multiprogramming

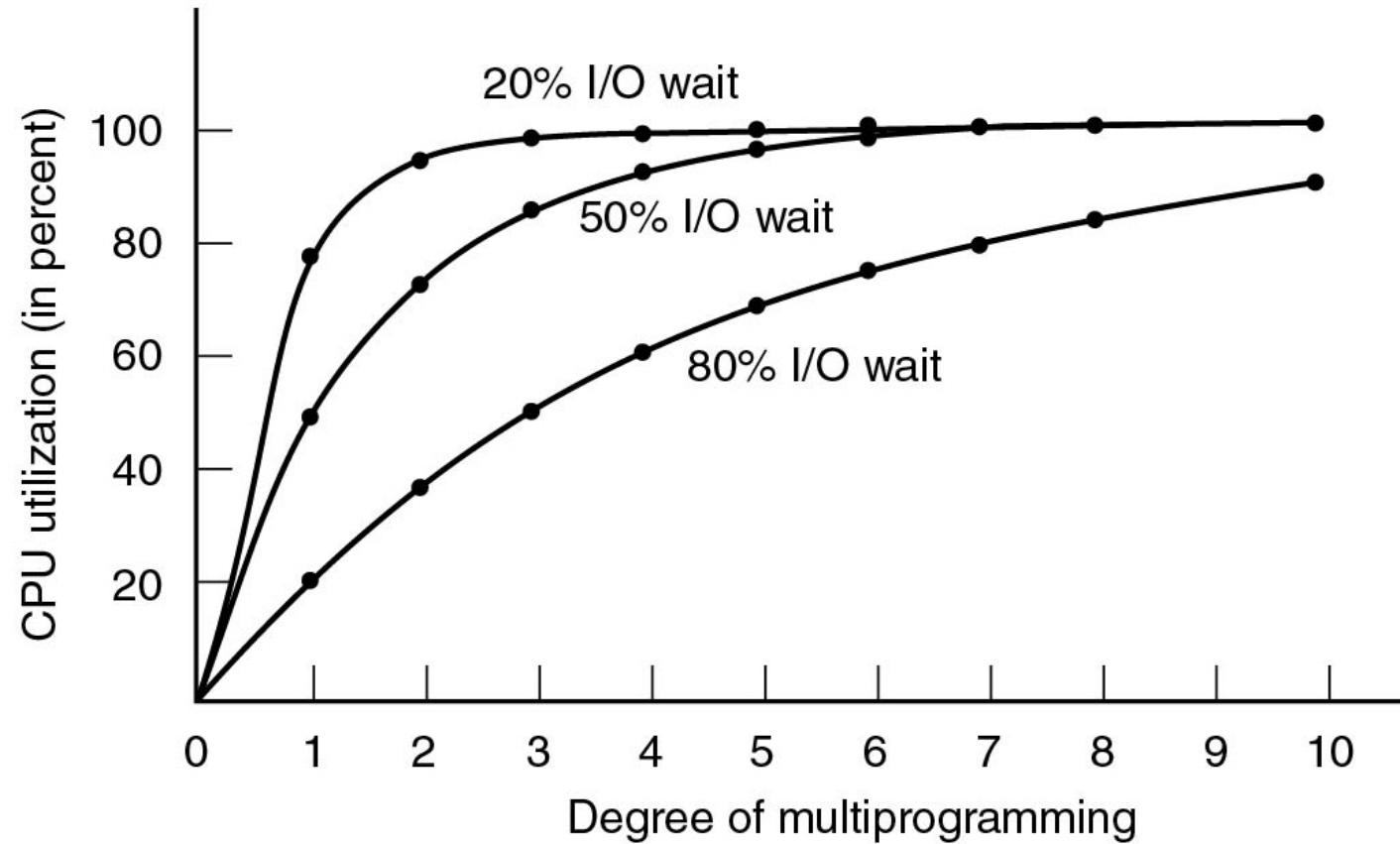


Figure 2.6 CPU utilization as a function of the number of processes in memory.

Signal Handling (1 of 2)

- Signal types:
 - Hardware-induced (e.g., SIGILL)
 - Software-induced (e.g., SIGQUIT or SIGPIPE)
- Actions:
 - **Term, Ign, Core, Stop, Cont**
 - Default action on per-signal basis, typically overridable
 - Signals can be typically blocked and actions delayed
- Catching signals:
 - Process registers signal handler
 - OS delivers signal and allows process to run handler
 - Current execution context needs to be saved/restored

Signal Handling (2 of 2)

- Kernel delivers signal
- Stops code currently executing
- Saves context
- Executes signal handling code
- Restores original context

Threads

Threads

- Implicit assumption so far:
 - 1 process $\rightarrow$ 1 thread of execution
- Multithreaded execution:
 - 1 process $\rightarrow$ N threads of execution
- Why allow multiple **threads** per process?
 - Lightweight processes
 - Allow space- and time- efficient parallelism
 - Organized in thread groups
 - Allow simple communication and synchronization

Thread Usage (1)

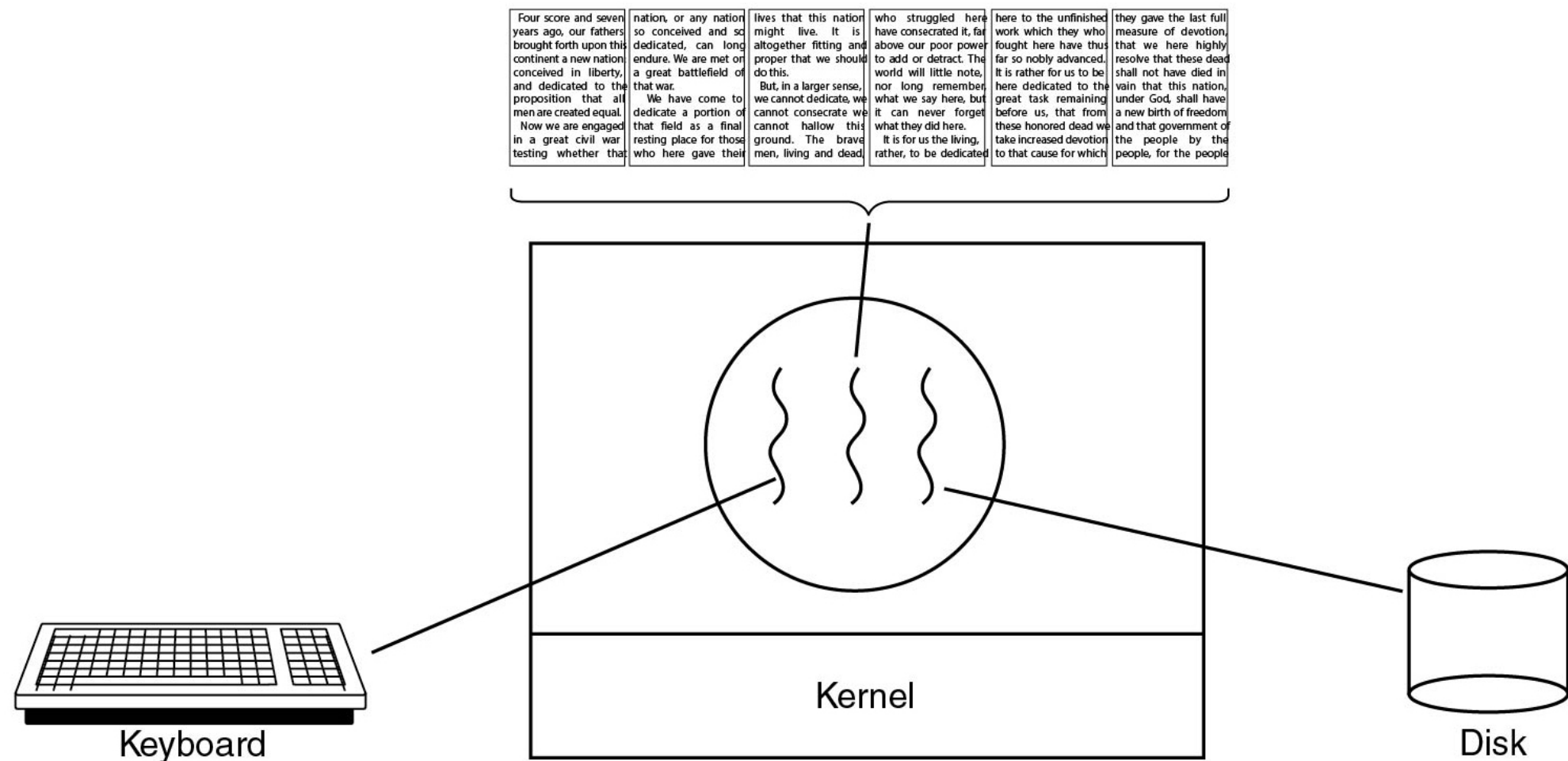


Figure 2.7 A word processor with three threads.

Thread Usage (2)

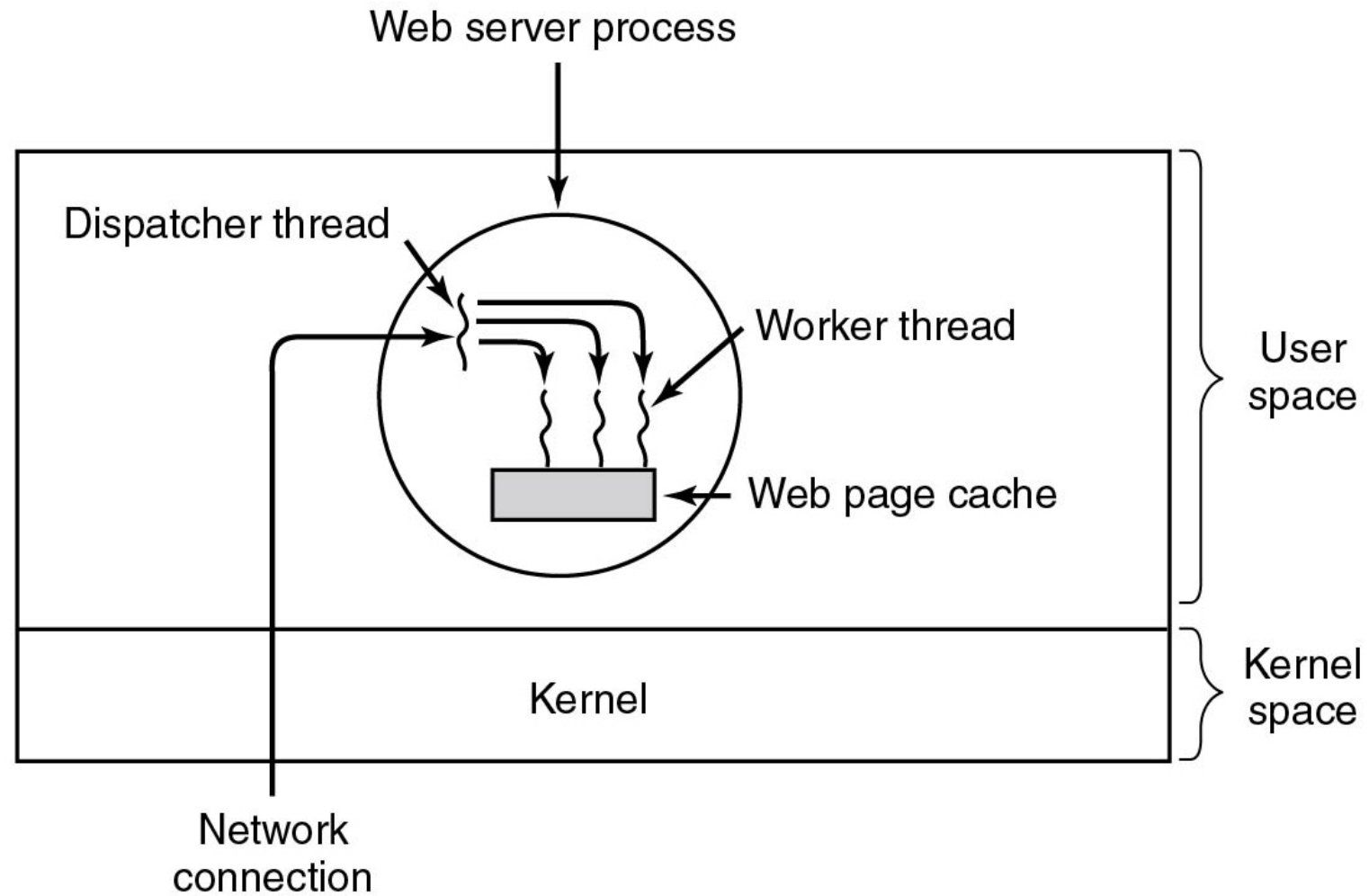


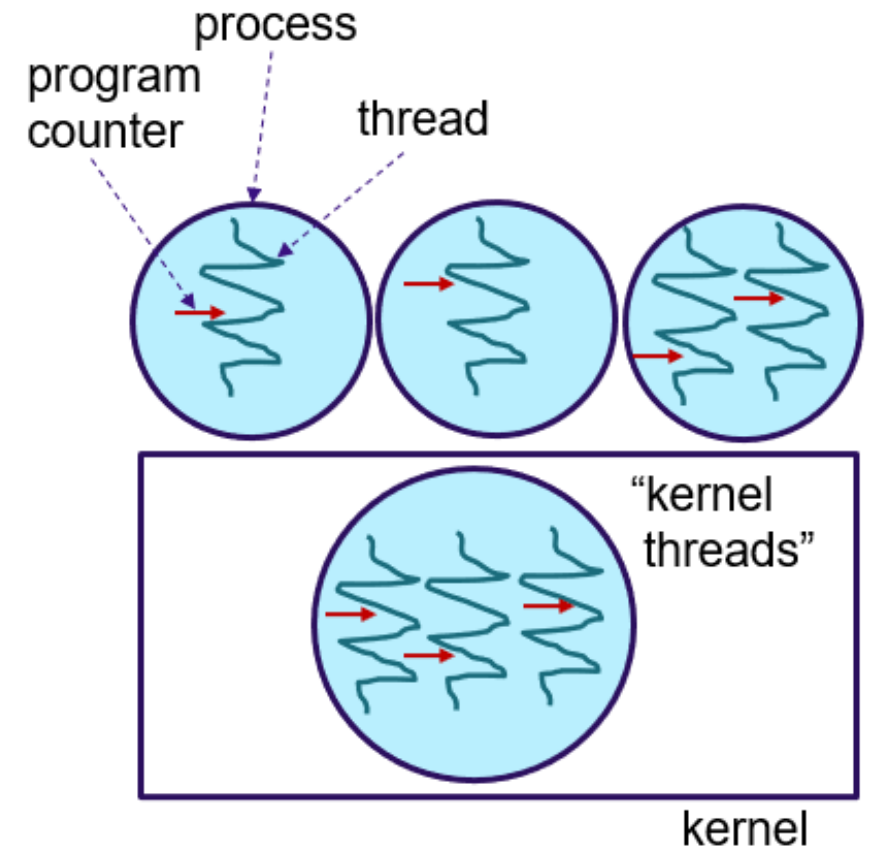
Figure 2.8 A multithreaded Web server.

Threads, Process, Finite State Machine

Model	Characteristics
Threads	Parallelism, blocking system calls
Single-threaded process	No parallelism, blocking system calls
Finite-state machine	Parallelism, nonblocking system calls, interrupts

Threads and Processes

- Threads reside in the same address space of a single process
- All information exchange is via data shared between the threads
- Threads synchronize via simple primitives
- Each thread has its own stack, hardware registers, and state
- Thread table/switch: a lighter process table/switch
- Each thread may call any OS-supported system call on behalf of the process to which it belongs



The Classical Thread Model (1)

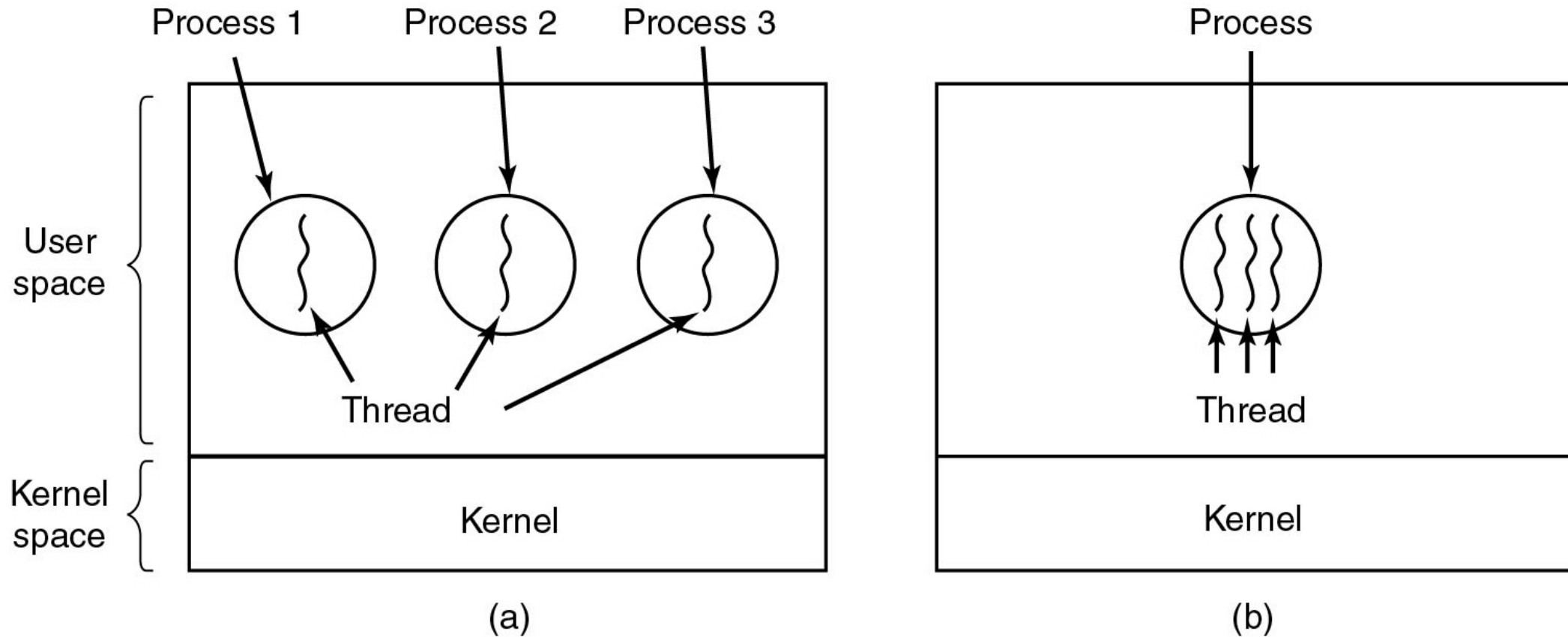


Figure 2.10 (a) Three processes each with one thread. (b) One process with three threads.

The Classical Thread Model (2)

Per process items	Per thread items
Address space Global variables Open files Child processes Pending alarms Signals and signal handlers Accounting information	Program counter Registers Stack State

Figure 2.11 The first column lists some items shared by all threads in a process. The second one lists some items private to each thread.

The Classical Thread Model (3)

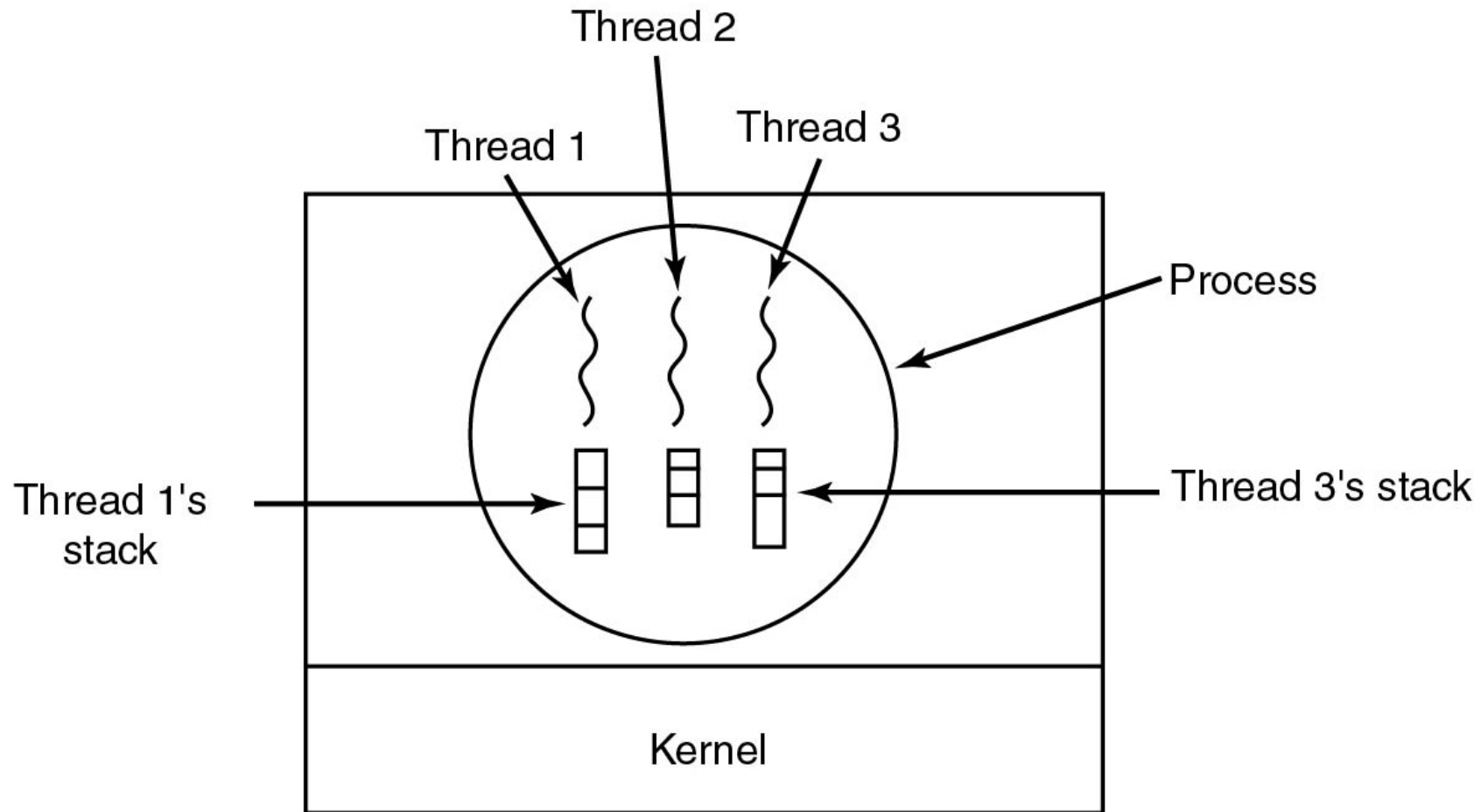
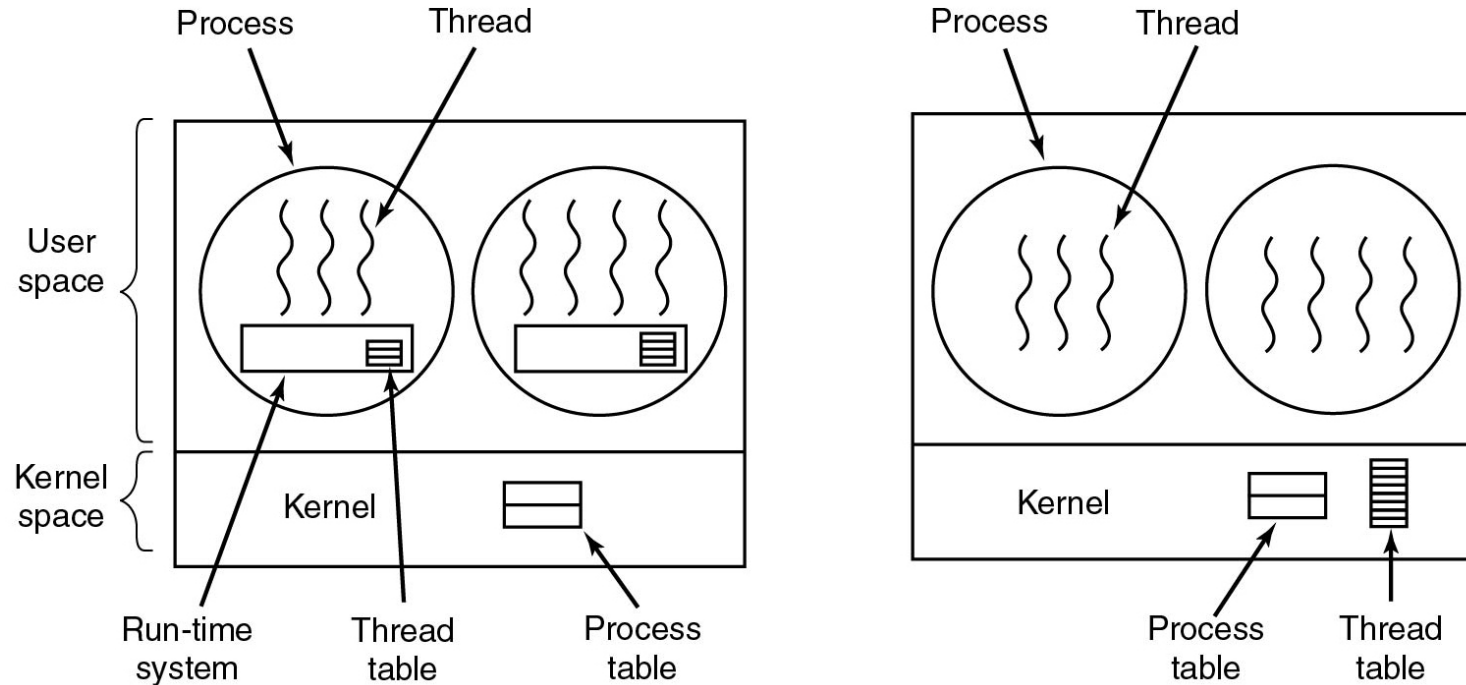


Figure 2.12 Each thread has its own stack.

User Threads: Pros and Cons



- + Thread switching time (no mode switch)
- + Scalability, customizability (no in-kernel management)
- Transparency (typically requires app cooperation)
- Parallelism (blocking syscalls are problematic)

Hybrid Implementations

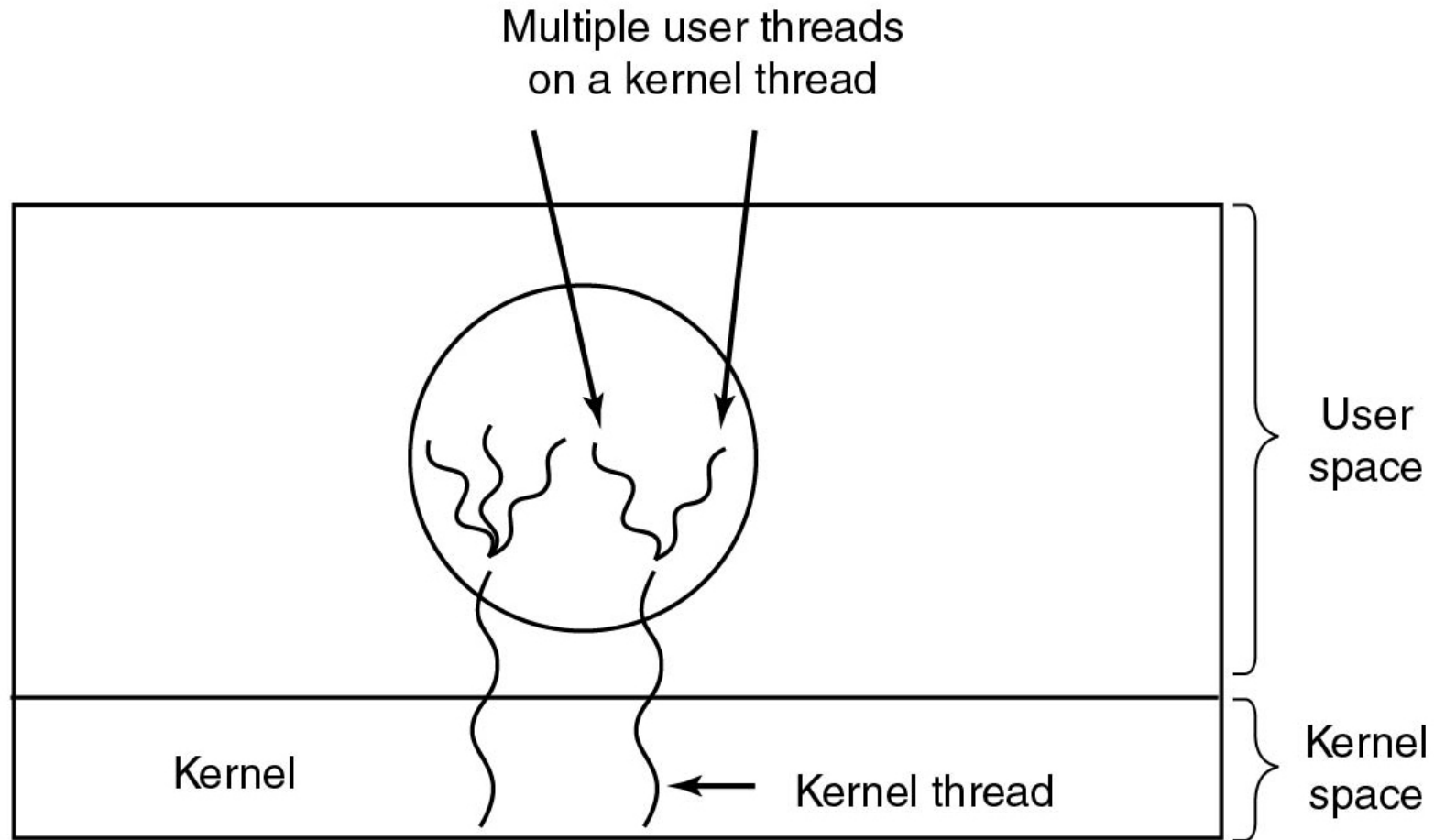


Figure 2.16 Multiplexing user-level threads onto kernel-level threads.

Making Single-Threaded Code Multithreaded (1)

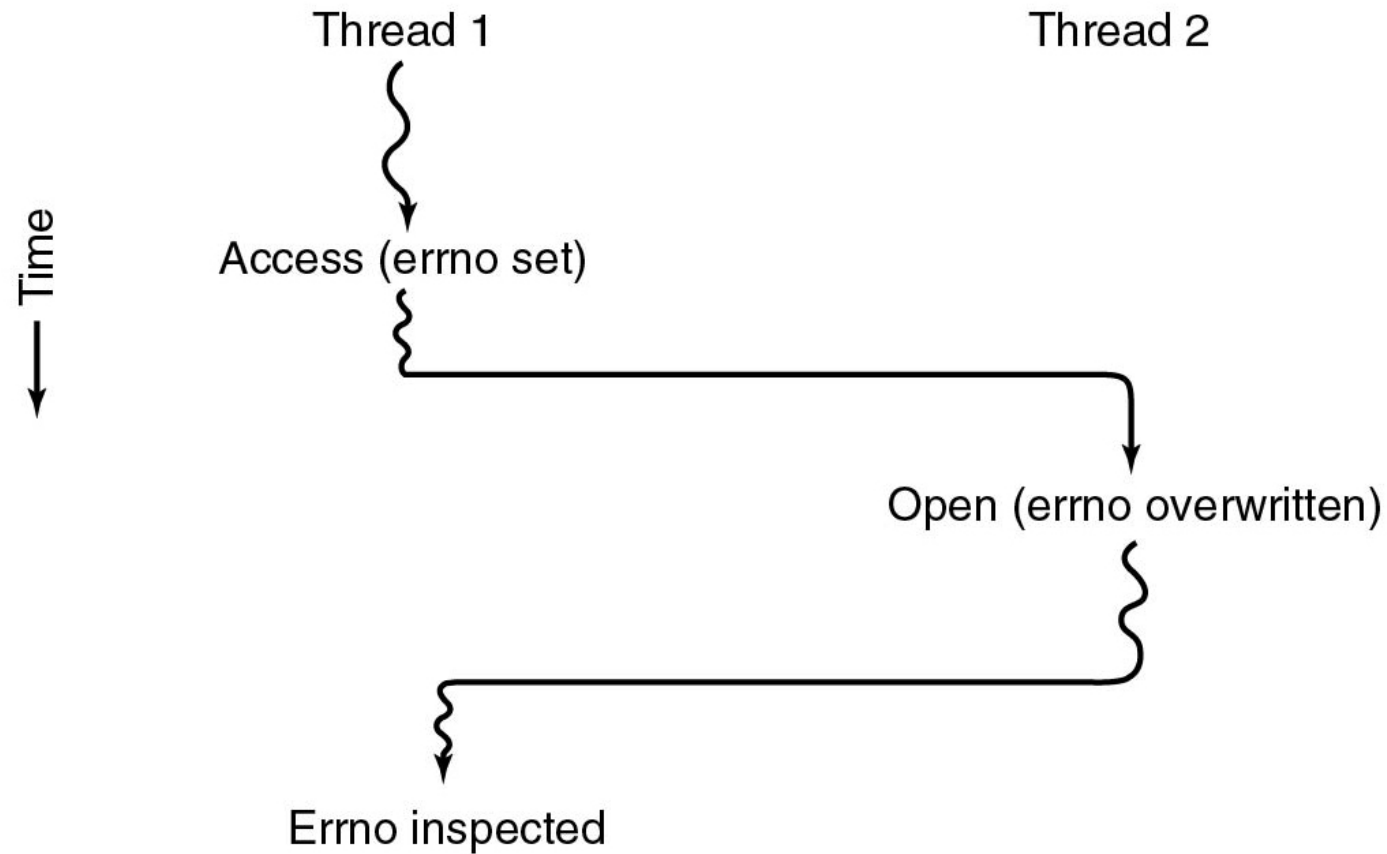


Figure 2.17 Conflicts between threads over the use of a global variable.

Making Single-Threaded Code Multithreaded (2)

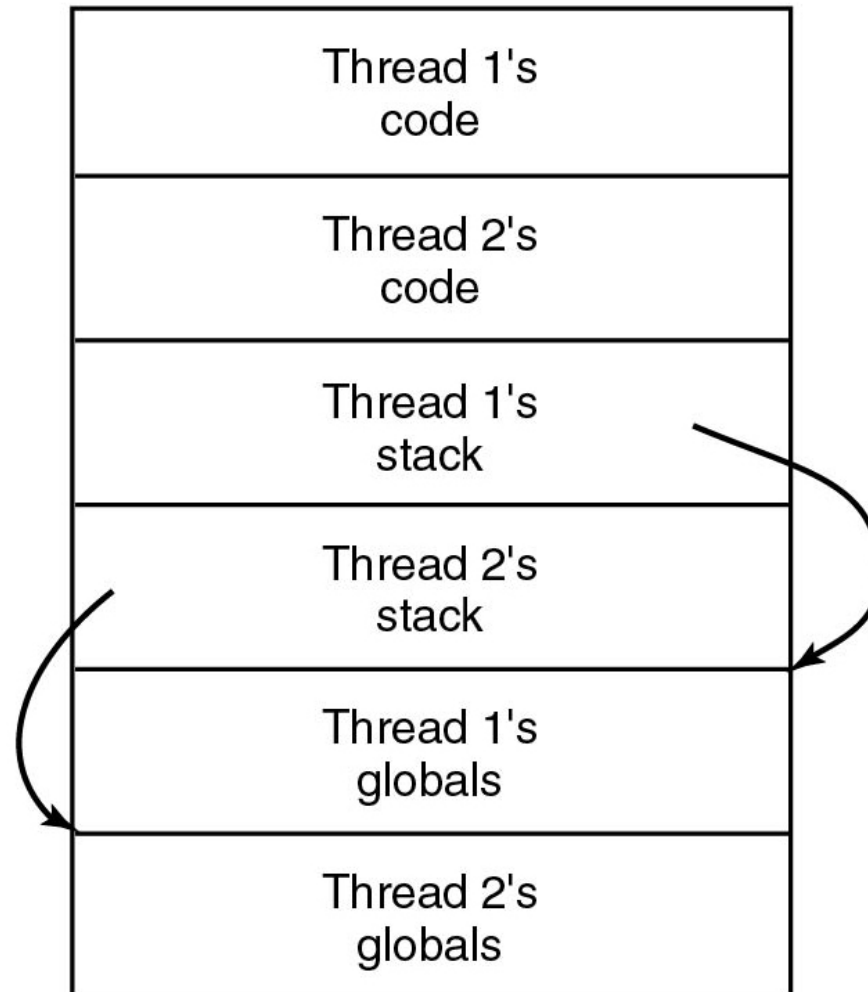


Figure 2.18 Threads can have private global variables.

Threads: Issues

- Does the OS keep track of threads?
 - Kernel threads vs. user threads
- What to do on fork?
 - Clone all threads vs. calling thread
 - What if a thread is currently blocking on a systems call?
- What to do with signals?
 - Send signal to all threads vs. single thread
 - Per-process or per-thread signal handlers
- Where to store per-thread variables?
- Does sharing come at a cost?
- Are threads required inside an operating system?

Event-Driven Servers

- Implement server as finite-state machine that responds to events using asynchronous system calls
 - E.g., the availability of data on a socket
- Implementation can be very efficient
- Every event leads to a burst of activity without blocking
- Most OS offer event notification interfaces for asynchronous I/O
 - Linux: epoll
 - FreeBSD: kqueue

Single-Threaded VS Multi-threaded VS Event-Driven Servers

Model	Characteristics
Threads	Parallelism, blocking system calls
Single-threaded process	No parallelism, blocking system calls
Finite-state machine	Parallelism, nonblocking system calls, interrupts

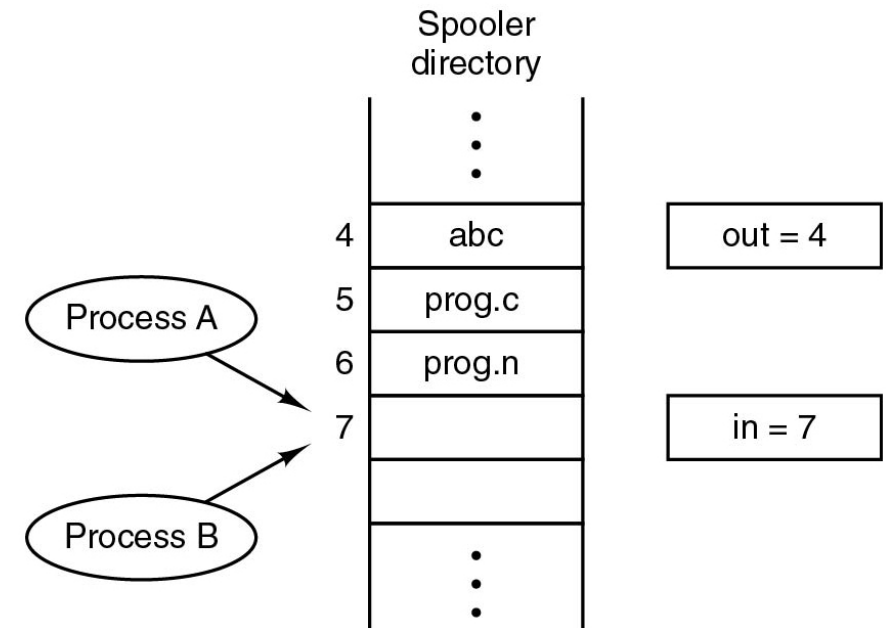
Figure 2.20 Three ways to construct a server.

Synchronization and Inter-Process Communication (IPC)

- Why?
- Processes need some way to **communicate**:
 - To share data throughout the execution
- No explicit cross-process sharing:
 - → Data must be normally exchanged between processes
- Processes need some way to **synchronize**:
 - To account for dependencies
 - To avoid they get in each other's way
 - Also applies to multithreaded execution

Race Conditions

- Process **A** reads $in=7$ and decides to append its file at that position
- **A** is suspended by OS (because its slott expired)
- Process **B** also reads $in=7$ and puts its file at that position
- **B** sets $in=8$ and eventually gets suspended.
- **A** writes its file to position 7
- **Problem:** reading/updating in should be an atomic action. If it is not, processes can race each other and come to wrong conclusions



Critical Regions (1)

Requirements to avoid race conditions:

1. No two processes may be simultaneously inside their critical regions.
2. No assumptions may be made about speeds or the number of CPUs.
3. No process running outside its critical region may block other processes.
4. No process should have to wait forever to enter its critical region.

Critical Regions (2)

- Critical region: a code region with access to shared resources
 - No two processes may be simultaneously in their critical regions
 - No assumptions may be made about speeds or nr. of CPUs
 - No process running outside its critical region may block others
 - No process should have to wait forever to enter its critical region
- (Non)solutions:
 - Disable interrupts: simply prevent that the CPU can be reallocated. Works for single-CPU systems only
 - Lock variables: guard critical regions with 0/1 variables. Races now occur on the lock variables themselves

Critical Regions (3)

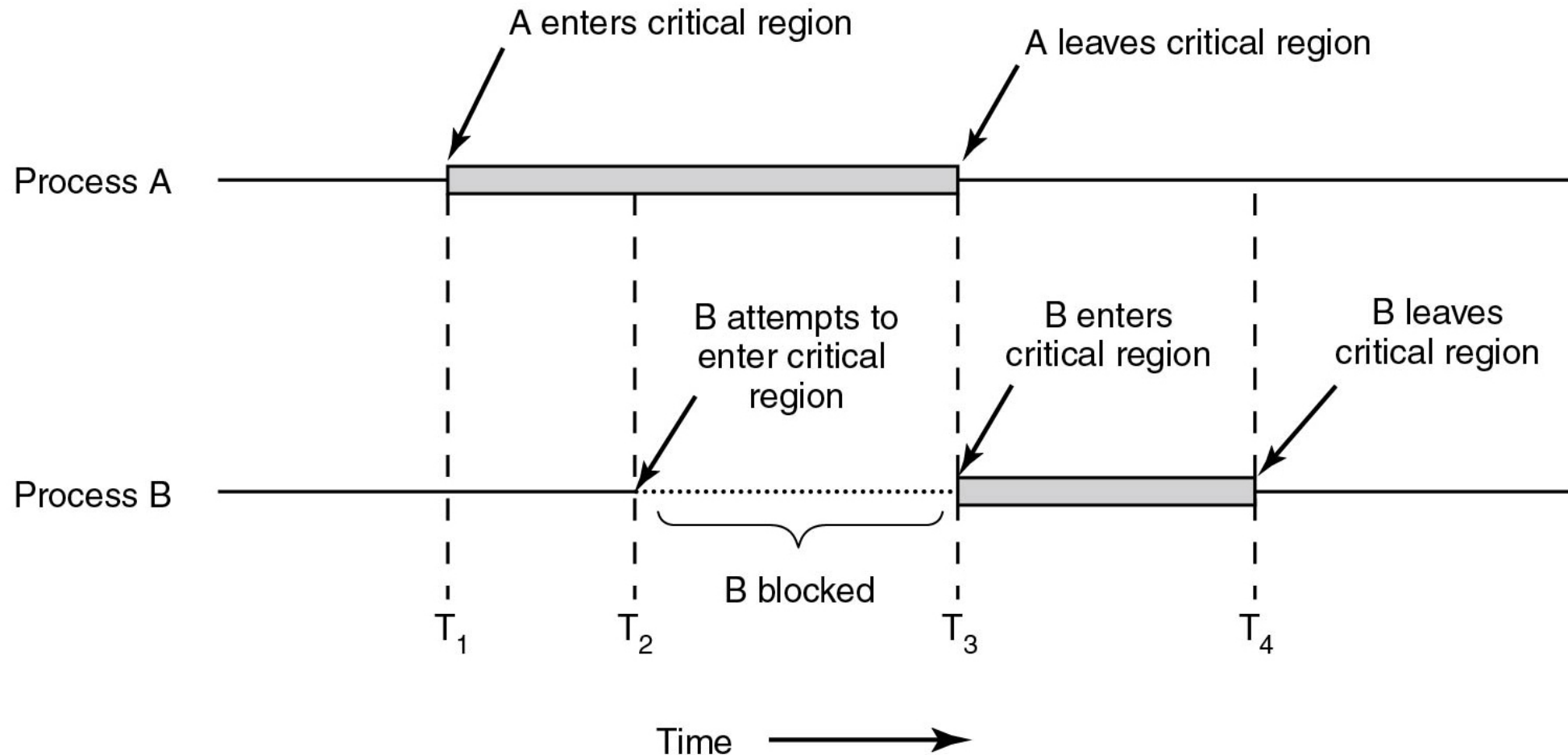


Figure 2.22 Mutual exclusion using critical regions.

Producer-Consumer (1 of 4)

https://raw.githubusercontent.com/xiaoyuansuo51-webster/COSC_2010_fall_2026/refs/heads/main/Animations/Producer_Consumer.html

Semaphores

- **Idea:** introduce a special sema integer type with 2 operations:
 - **down:**
 - If $\text{sema} \leq 0$ then block the calling process
 - $\text{sema} = \text{sema} - 1$ otherwise
 - **up:**
 - if there is a process blocking on sema, wake it up
 - $\text{sema} = \text{sema} + 1$ otherwise
- OS guarantees all the operations are **atomic** by design
 - **Disable interrupts** on single processors
 - **Spin locking** on multiprocessors
- Back to busy waiting problems?
- Uses for binary semaphores (aka **mutexes**)?

Semaphores: Producer-Consumer (1 of 5)

https://raw.githubusercontent.com/xiaoyuansuo51-webster/COSC_2010_fall_2026/refs/heads/main/Animations/semaphore-producer-consumer-animation.html

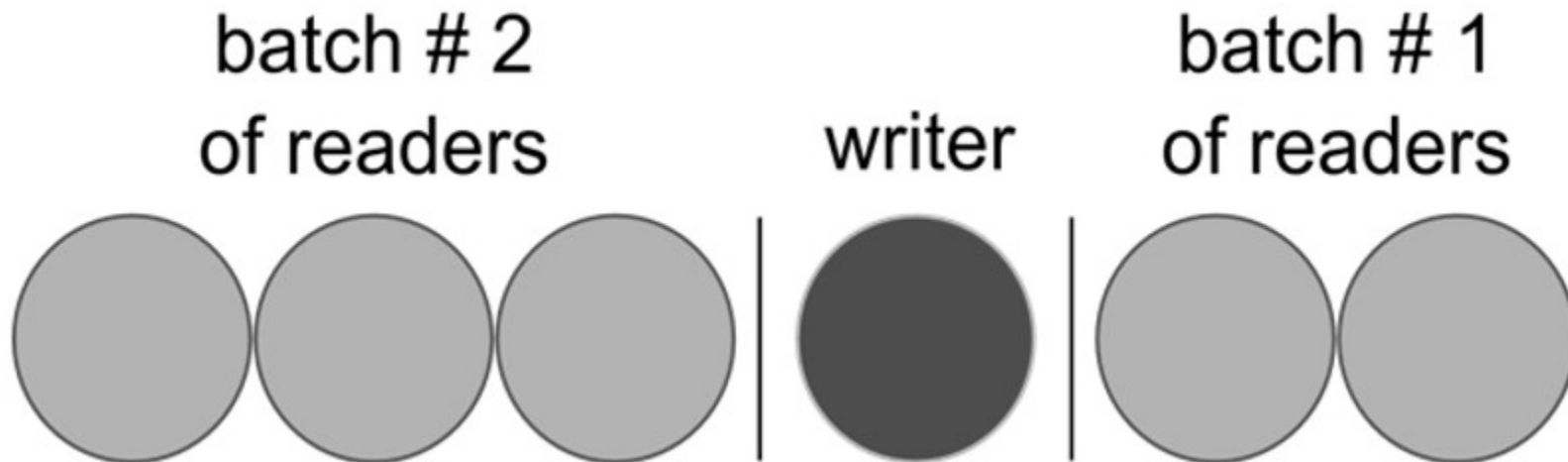
Readers/Writers

- N processes access (i.e., read or write) some shared data
- At any given time: R readers **or** 1 writer allowed.

https://raw.githubusercontent.com/xiaoyuansuo51-webster/COSC_2010_fall_2026/refs/heads/main/Animations/Producer_Consumer.html

Readers/Writers

- **Idea:** Build a queue of readers and writers
- Let several readers in at the same time
- Allow 1 writer when no readers are active
- How long may the writer have to wait?



Mutexes: Simple Implementation

https://raw.githubusercontent.com/xiaoyuansuo51-webster/COSC_2010_fall_2026/refs/heads/main/Animations/mutex-tsl-animation.html

Figure 2.30 Implementation of **mutex_lock** and **mutex_unlock**.

Monitors

- Semaphores have been heavily criticized for the chaos they can introduce
- Monitors: more structured approach towards process synchronization:
 - Serialize the procedure calls on a given module
 - Use condition variables to wait / signal processes
- Monitors require language support
- Popular in managed languages, e.g., Java:
 - Synchronized methods / blocks
 - Wait, notify, notifyall primitives

Monitors (1)

- https://raw.githubusercontent.com/xiaoyuansuo51-webster/COSC_2010_fall_2026/refs/heads/main/Animations/monitor-producer-consumer-animation.html

Message Passing

- Solution to both the process **synchronization** and the process **communication** problems
- Most common choice in **multiserver** OS designs
- Processes interact by sending and receiving **messages**:
 - **send**(destination, &message);
 - **receive**(source, &message);
 - **receive**(ANY, &message);

Message Passing: Producer-Consumer (1 of 5)

https://raw.githubusercontent.com/xiaoyuansuo51-webster/COSC_2010_fall_2026/refs/heads/main/Animations/message-passing-producer-consumer-animation.html

Synchronization Techniques: Comparison (1)

Technique	Main idea	Waiting behavior	Efficiency	Best suited for	Real applications	Main limitation
Sleep/Wakeup	A blocked process sleeps until another process explicitly wakes it	Blocking	Efficient when implemented correctly	Basic process coordination	OS process scheduling and device-event handling	Can suffer from lost wakeups if checking and sleeping are not atomic
Semaphore	An integer counter is modified atomically using wait/down and signal/up	Usually blocking	Good; avoids busy waiting	Resource counting and process ordering	Bounded buffers, connection pools, OS kernels	Error-prone; incorrect operation order can cause deadlock
Mutex	Only one thread at a time may enter a critical section	Blocking, yielding, or spinning	Very efficient for short critical sections	Protecting one shared resource	Shared counters, data structures, file metadata	Does not directly express conditions such as “buffer not empty”
Monitor	Encapsulates shared data, mutual exclusion, and condition variables	Blocking	Good; synchronization is largely automatic	Structured shared-memory programming	Java synchronized, C# lock, thread-safe classes	Requires language/runtime support; incorrect condition logic is still possible
Message Passing	Processes exchange data through send and receive instead of directly sharing memory	Blocking or nonblocking	Good for distributed systems; communication overhead may be significant	Processes without shared memory	MPI, sockets, microservices, actor systems	Data copying, latency, message ordering, and communication failures
Reader–Writer Lock	Multiple readers may enter together, but a writer requires exclusive access	Blocking	Excellent for read-heavy workloads	Shared data that is read much more often than written	Databases, caches, file systems, configuration data	Reader or writer starvation may occur

Overall Comparison

Question	Sleep/Wakeup	Semaphore	Mutex	Monitor	Message Passing	Reader–Writer Lock
Shared memory required?	Usually	Usually	Yes	Yes	No	Yes
Supports resource counting?	No	Yes	No	Through state variables	Through queued messages	No
Mutual exclusion built in?	No	Possible	Yes	Yes	Avoids direct sharing	Yes
Suitable across machines?	No	Usually no	No	Usually no	Yes	No
Programming difficulty	High	Medium–high	Low–medium	Low–medium	Medium–high	Medium
Major risk	Lost wakeup	Deadlock	Deadlock	Incorrect condition use	Communication failure	Starvation

Barriers

No process may proceed into the next phase until all processes are ready to proceed to the next phase

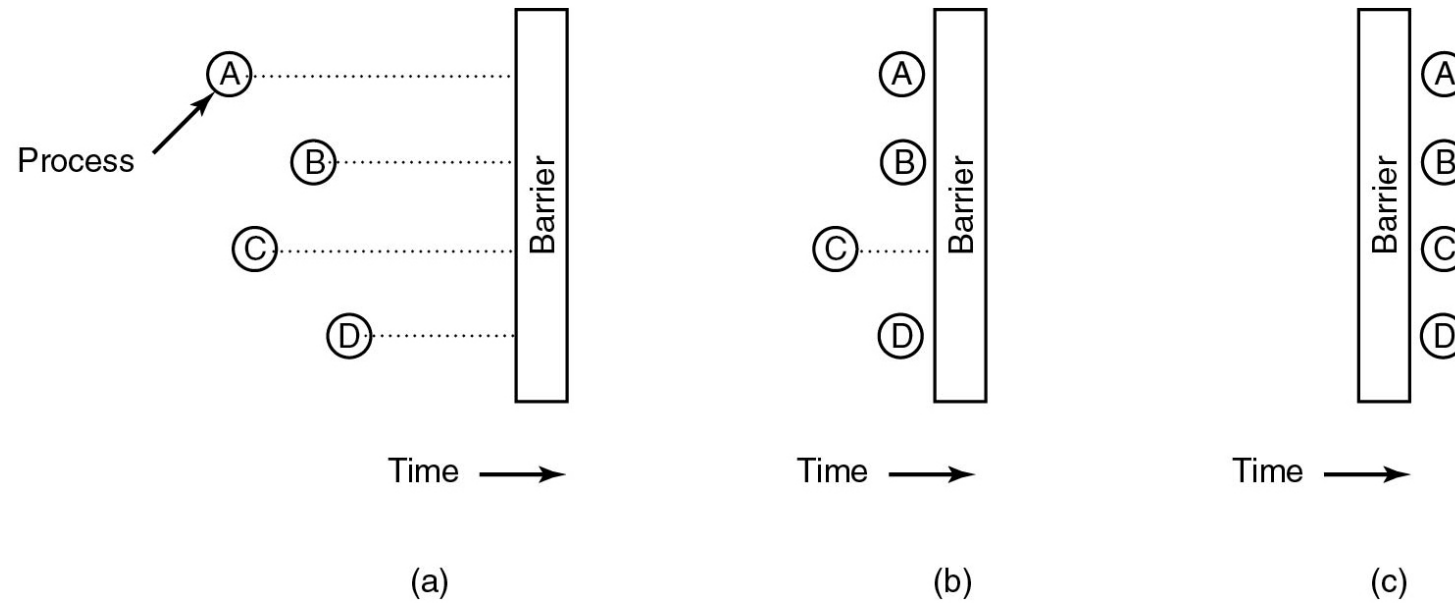


Figure 2.38 Use of a barrier. (a) Processes approaching a barrier. (b) All processes but one blocked at the barrier. (c) When the last process arrives at the barrier, all of them are let through.

Priority Inversion (1)

Consider the Mars Pathfinder

- It had three subsystems
 - A data-distribution system (**High Prio**)
 - A Communication system (**Med Prio**)
 - A meteorological data gathering (**Low Prio**)

There was also a common hardware subsystem:

The data bus used by Task 1 and 3.

It is protected by a mutex

Question: What can happen?



NASA/JPL-Caltech

Priority Inversion (2)

Consider the Mars Pathfinder

- It had three subsystems
 - A data-distribution system (**High Prio**)
 - A Communication system (**Med Prio**)
 - A meteorological data gathering (**Low Prio**)

There was also a common hardware subsystem:
The data bus used by Task 1 and 3.

It is protected by a mutex

Question: What can happen?

Imagine the following scenario: (1) Low Prio task takes mutex, (2) gets interrupted by Med Prio task, (3) which is interrupted by High Prio task (which blocks because Low Prio task holds lock) Medium Prio task runs even though there is a High Prio task to run (priority is inverted)



NASA/JPL-Caltech

Priority Inversion (3)

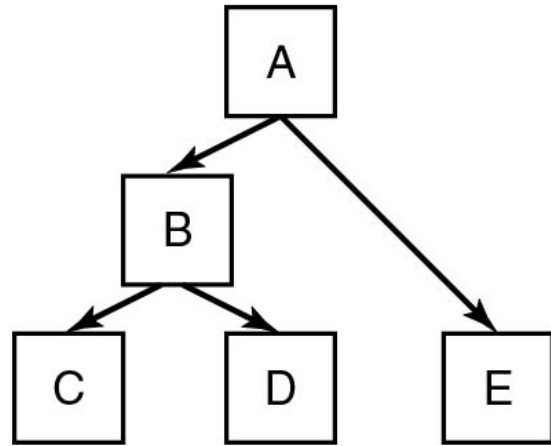
- Several methods to solve priority inversion
 - Disable all interrupts while in the critical region
 - Priority ceiling: associate a priority with the mutex and assign that to the process holding it
 - Priority inheritance: A low-priority task holding the mutex temporarily inherits the priority of the high-priority task trying to obtain it
 - Random boosting: randomly assigning mutex-holding threads a high priority until they exit the critical region

Avoiding Locks: Read-Copy-Update (1)

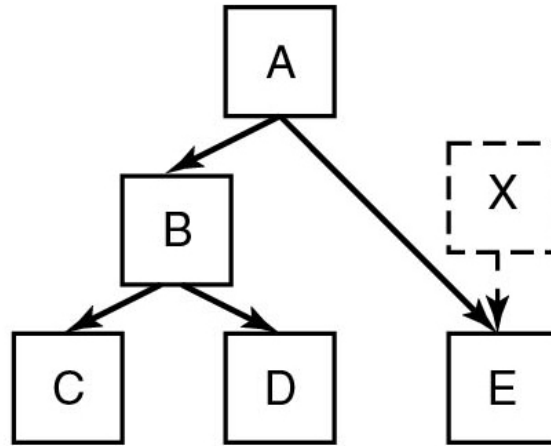
- An instance of **relativistic programming**
- Do not try to avoid conflicts between readers and writers
- Tolerate them and ensure a correct result regardless of the order of events
- Allow writer to update data structure even if other processes are still using it.
 - Parallel copies of a data structure.
 - Ensure that each reader either reads the old or the new version, but not some weird combination of the 2.
 - Single-pointer readers/writers scheme
- Readers execute in read-side sections
- Writer operates 3 steps:
 - Atomically update pointer to new copy
 - Wait for existing readers (**grace period**)
 - Reclaim old copy

Avoiding Locks: Read-Copy-Update (2)

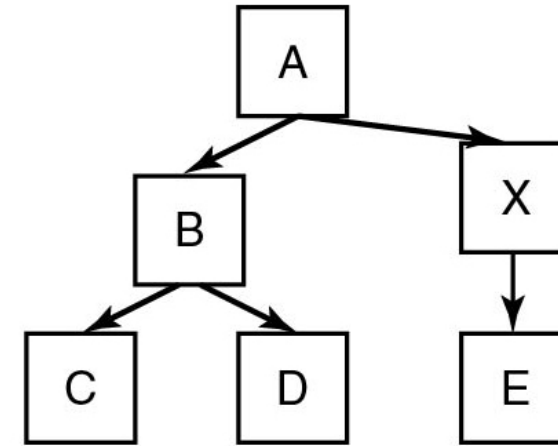
Adding a node:



(a) Original tree.



(b) Initialize node X and connect E to X. Any readers in A and E are not affected.

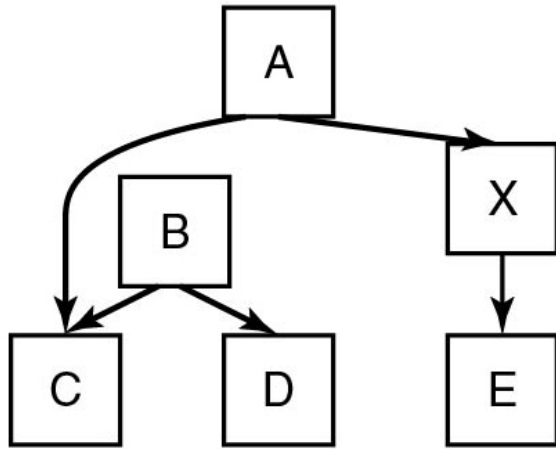


(c) When X is completely initialized, connect X to A. Readers currently in E will have read the old version, while readers in A will pick up the new version of the tree.

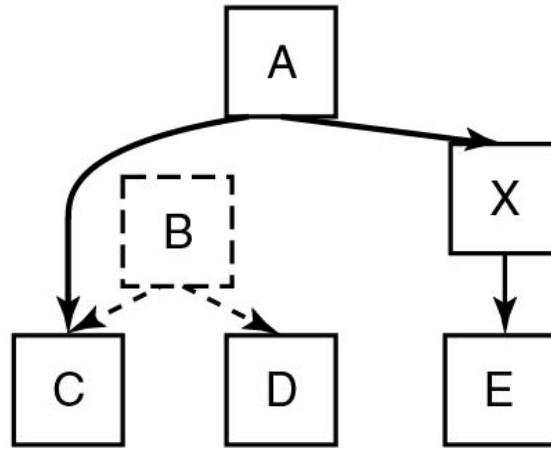
Figure 2.39 Read-Copy-Update: inserting a node in the tree and then removing a branch-all without locks

Avoiding Locks: Read-Copy-Update (3)

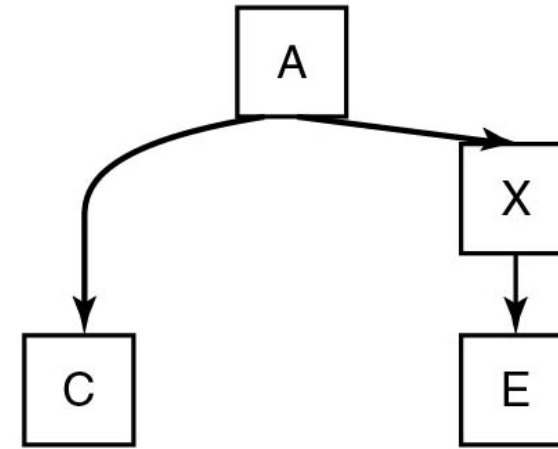
Removing nodes:



(d) Decouple B from A. Note that there may still be readers in B. All readers in B will see the old version of the tree, while all readers currently in A will see the new version.



(e) Wait until we are sure that all readers have left B and C. These nodes cannot be accessed any more.



(f) Now we can safely remove B and D

Figure 2.39 Read-Copy-Update: inserting a node in the tree and then removing a branch-all without locks

Synchronization Techniques: Comparison (2)

Technique	Main idea	Waiting behavior	Efficiency	Best suited for	Real applications	Main limitation
Spinlock	A thread repeatedly checks a lock until it becomes available	Busy waiting	Excellent for extremely short waits; poor for long waits	Short kernel-level critical sections	Multiprocessor kernels and low-level runtime code	Wastes CPU time while waiting
Condition Variable	A thread waits until a condition becomes true; another thread signals a state change	Blocking	Efficient when paired with a mutex	Waiting for state changes	Thread pools, queues, monitors, producer–consumer	Must be used with a mutex and normally checked in a while loop
Barrier	All participating threads must reach the same point before any continue	Blocking	Efficient for phase-based parallel work	Algorithms divided into synchronized stages	OpenMP loops, simulations, matrix algorithms	A delayed or failed participant holds up everyone
Atomic Operation	Performs a small read–modify–write operation indivisibly	Usually nonblocking	Very efficient under low contention	Simple shared counters, flags, and lock-free structures	Atomic counters, reference counting, lock implementation	Difficult to use for complex operations; contention still reduces performance
Read–Copy–Update (RCU)	Readers access data without locking; writers publish a new version and delay reclamation	Readers rarely block	Extremely efficient for read-mostly workloads	Very frequent reads and rare updates	Linux kernel routing and process tables	Complex update and memory-reclamation rules
Transactional Memory	A group of memory operations commits atomically or retries after a conflict	Optimistic retry	Good when conflicts are uncommon	Complex critical sections	Hardware transactions and research/runtime systems	Performance becomes unpredictable under high contention

Scheduling

Introduction to Scheduling Process Behavior

CPU bound vs. I/O bound processes

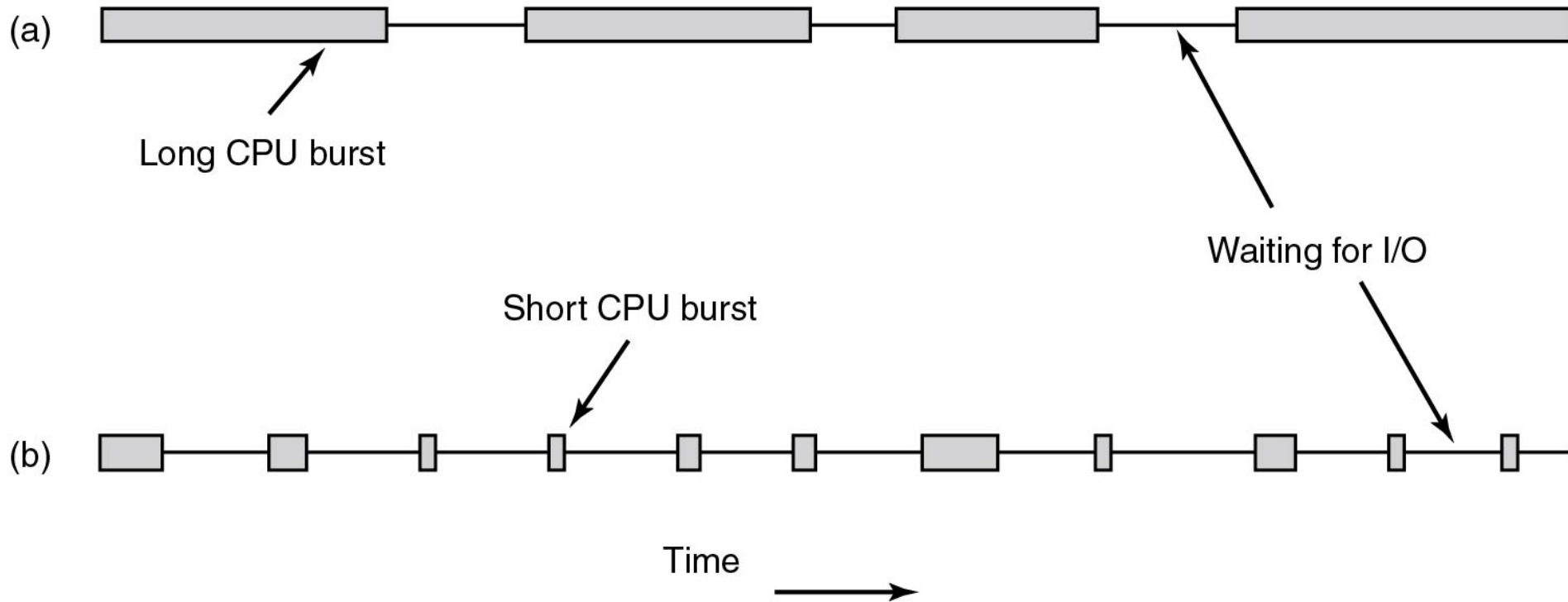
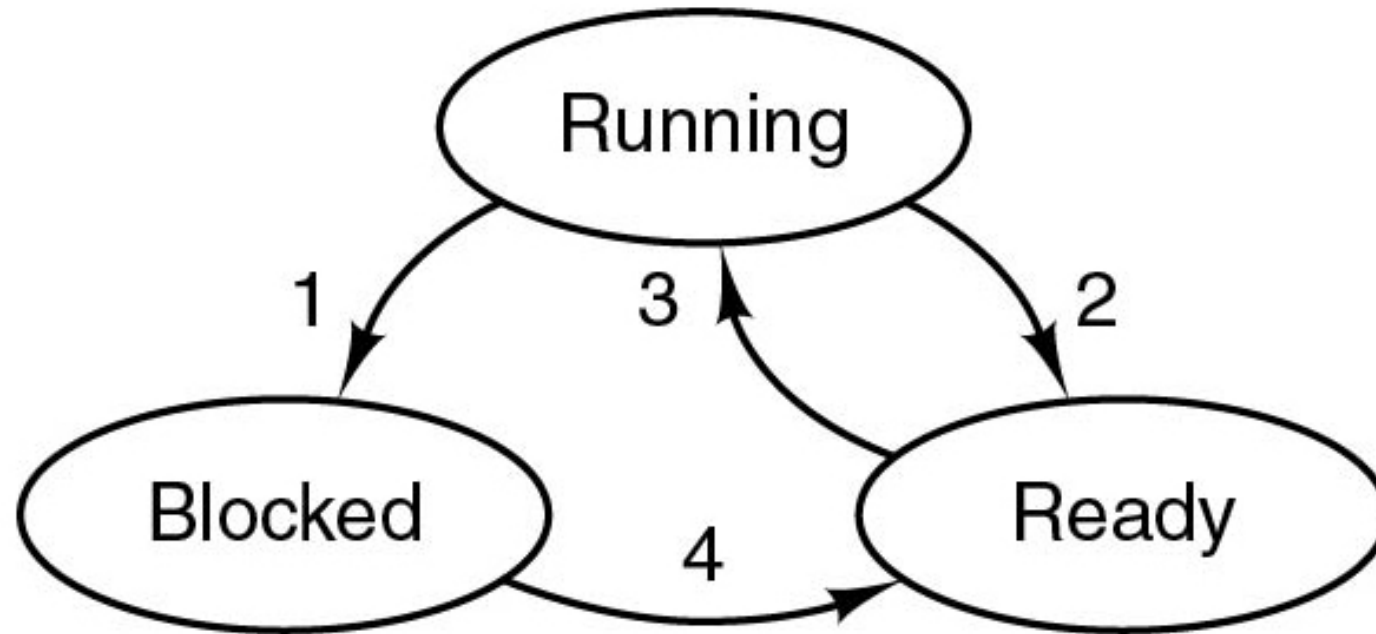


Figure 2.40 Bursts of CPU usage alternate with periods of waiting for I/O.
(a) A CPU-bound process. (b) An I/O-bound process.

Process State Revisited



If more processes ready than CPUs available:

- **Scheduler** decides which process to run next
- Algorithm used by scheduler is called **scheduling algorithm**

When to Schedule?

- ***Process exits***
- ***Process blocks on I/O, Semaphore, etc.***
- *When a new process is created*
- *When an interrupt occurs:*
 - *I/O, clock, syscall, etc.*

Preemptive vs **non-preemptive** scheduling?

Categories of Scheduling Algorithms

- Batch
- Interactive
- Real time

Scheduling Algorithm Goals (1 of 2)

- Different goals for different systems
 - Batch
 - Interactive
 - Real time
- All systems:
 - Fairness - giving each process a fair share of the CPU
 - Policy enforcement - seeing that stated policy is carried out
 - Balance - keeping all parts of the system busy

Batch Systems



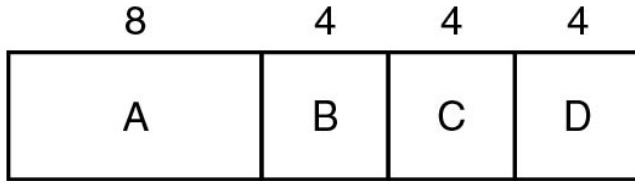
Scanrail/123RF

- Throughput : maximize jobs per hour
- Turnaround time : minimize time between submission and termination
- CPU utilization : keeping the CPU busy all the time

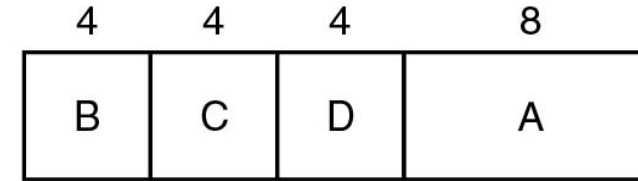
First-Come First-Served

- Process jobs in order of their arrival
- Non-preemptive
- Single Process Queue
 - New jobs or blocking processes are added to the end of the queue
- “Convoy Effect” if only few CPU bound and many I/O bound processes

Shortest Job First (1 of 2)



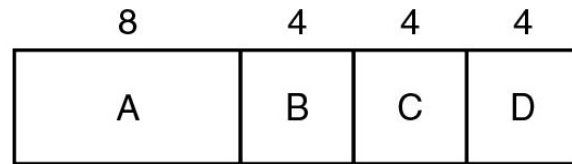
(a)



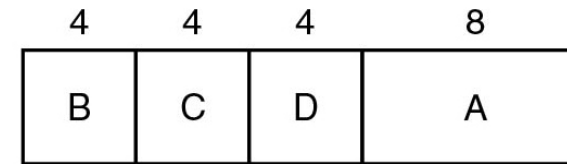
(b)

- Pick the job with the shortest run time
- Provably optimal:
 - Lowest turnaround time

Shortest Job First (2 of 2)



(a)



(b)

- Pick the job with the shortest run time
- Provably optimal:
 - Lowest turnaround time
 - Only if all jobs are available simultaneously
 - If new jobs arrive, it may lead to starvation
- Runtimes have to be known in advance
- Highest-response-ratio-next
 - Improved version of Shortest Job First

Interactive Systems

- **Response time:** respond to requests quickly
- **Proportionality:** meet users' expectations



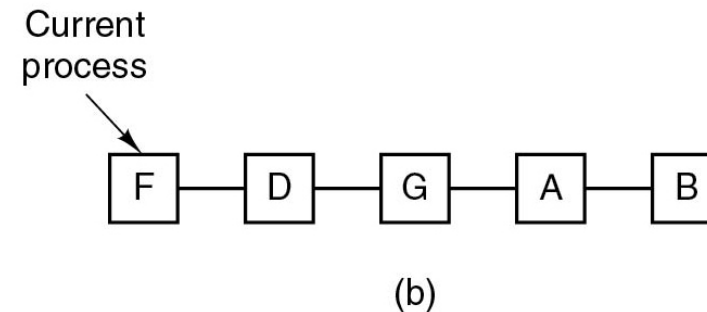
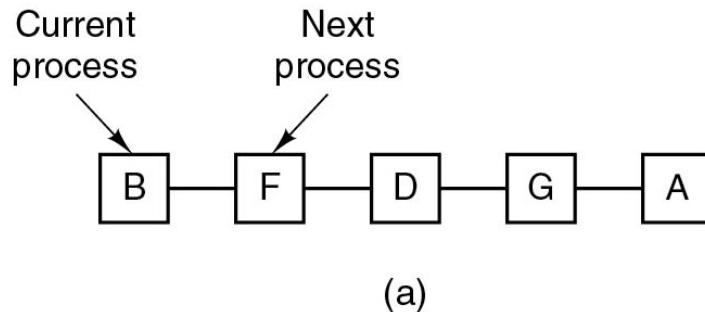
McClatchy-Tribune/Tribune Content Agency LLC/Alamy Stock Photo

Apple MacIntosh (1984)

Round Robin Scheduling

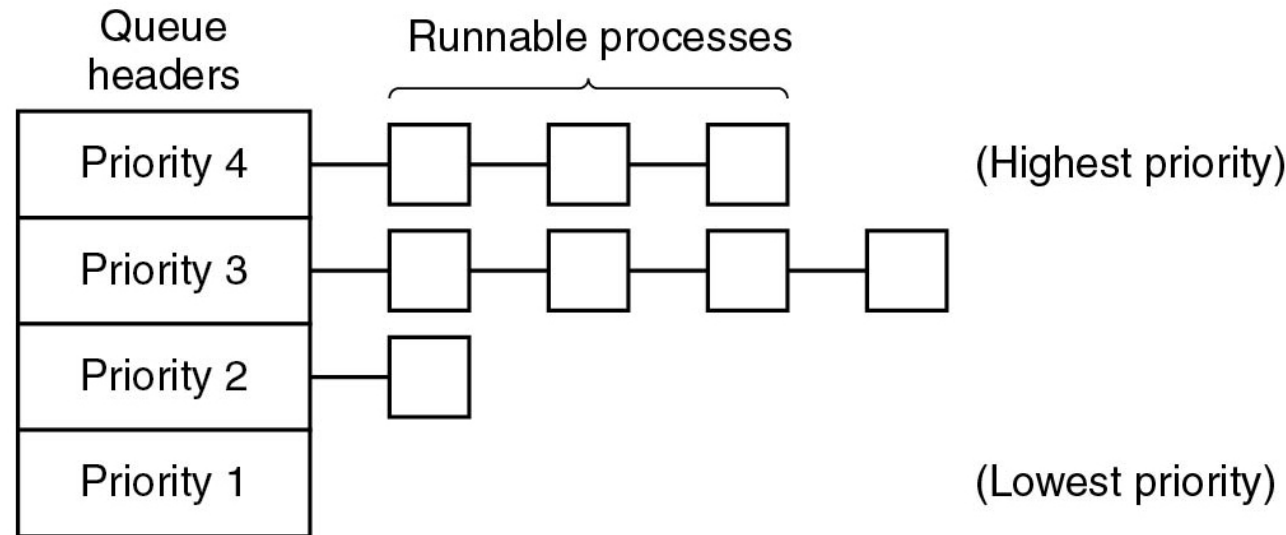
- Preemptive scheduling algorithm
- Each process gets a **time slice** or **quantum**
- If process is still running at end of quantum it gets preempted and goes to end of ready queue
- Question: How big should the quantum be?

CPU utilization vs. response time



Priority Scheduling (1 of 2)

Simplest, multiple queues



- Similar to round robin but several ready queues
- Next process is picked from queue with highest priority
- Static vs. dynamic priorities
- What processes should have high priorities?

Priorities Are Complicated

and may not work out the way you think...
(remember priority inversion?)

Shortest Process Next

Problem: How to minimize response time for each priority queue?

Idea: Use shortest “job” first and try to best predict next running time



whatever runs between the waits

Solution: Form weighted average of previous running times of process → **Aging**

$$T_{k+1} = a * T_{k-1} + (1 - a) * T_k$$

Easy to implement when $a = 1/2$

Guaranteed Scheduling

Idea: N processes running → each process gets 1/Nth of CPU time
(also known as fair-share)

Solution:

- Calculate how much CPU time it might have gotten:
 - Time since process creation divided by N
- Measure actual consumed CPU time and form ratio
- 0.5 → process running half the time it was entitled to
- 2.0 → process running twice as much as it was entitled to
- Pick process with the smallest ratio to run next
- How to incorporate priorities (See Linux' CFS)?
- Note: fair-share scheduling can be per-user/-process

Lottery Scheduling

- Processes get lottery tickets
- Whenever a scheduling decision is made OS chooses a winning ticket randomly
- Processes can possess multiple tickets → Priorities
- Tickets can be traded between processes.
- Tickets are immediately available to newly created processes.

Policy vs. Mechanism

Important principle

Here: we may have a scheduling algorithm, but parameters to be filled in by user (process)

For instance, to give some child processes higher priority than others

Real Time Systems (1 of 3)



Yauhen_D/Shutterstock



NASA

- Meeting deadlines: avoid losing data
- Predictability: avoid quality degradation in multimedia systems

Real Time Systems (2 of 3)

- Systems where timing plays essential role
- Soft real time vs. Hard real time
- Can consist of periodic and aperiodic tasks
- Schedules can be static or dynamic
- System with periodic tasks is schedulable when:

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq 1$$

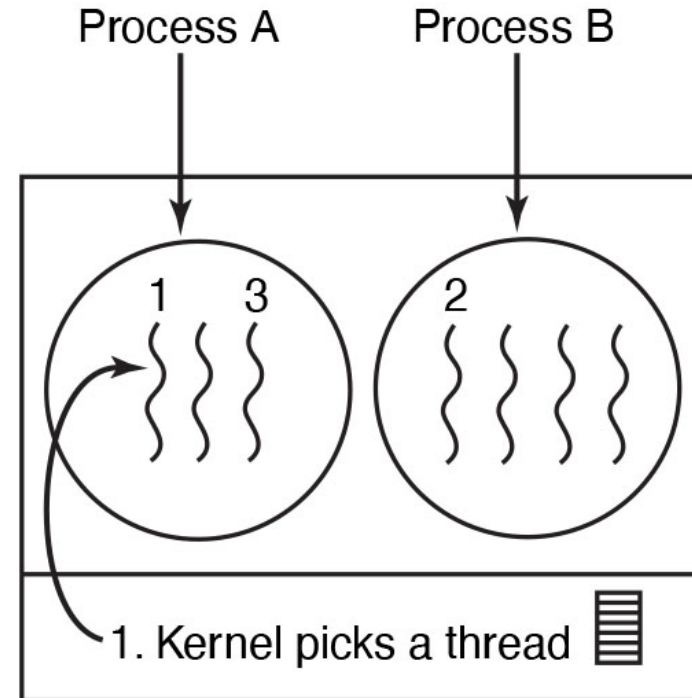
Real Time Systems (3 of 3)

Consider three jobs:

	Period	Req. CPU time
P1	100 ms	50 ms
P2	200 ms	30 ms
P3	500 ms	100 ms

Is this system schedulable?

Thread Scheduling (2)



Possible: A1, A2, A3, A1, A2, A3

Also possible: A1, B1, A2, B2, A3, B3

(b)

Figure 2.45 (b) Possible scheduling of kernel-level threads with the same characteristics as (a).

Basic CPU Scheduling Algorithms

Scheduling method	Main idea	Preemptive?	Efficiency / performance	Major problem	Best use or application
First-Come, First-Served (FCFS/FIFO)	Run processes in arrival order	No	Very low overhead; poor average waiting and response time	Convoy effect: short jobs wait behind long jobs	Simple batch systems and background queues
Shortest Process Next (SPN/SJF)	Run the ready process with the shortest estimated CPU burst	No	Minimizes average waiting time when burst lengths are known	Burst length is difficult to predict; long jobs may starve	Batch workloads with predictable execution times
Shortest Remaining Time First (SRTF)	Always run the process with the least remaining execution time	Yes	Excellent average waiting and turnaround time	Starvation and frequent preemption; requires runtime estimates	Systems emphasizing short-job completion
Round Robin (RR)	Give each process one time quantum in cyclic order	Yes	Good response and fairness; moderate context-switch overhead	Performance depends heavily on quantum size	Interactive and time-sharing systems
Priority Scheduling	Run the highest-priority ready process	Either	Fast response for important work	Low-priority processes may starve; priority inversion may occur	Operating systems with differentiated task importance
Highest Response Ratio Next (HRRN)	Choose the largest ratio: (waiting time + service time) / service time	No	Good balance between short jobs and long-waiting jobs	Requires service-time estimates; less suitable for interactive systems	Batch systems requiring reduced starvation

Advanced and Fair-Share Scheduling

Scheduling method	Main idea	Preemptive?	Efficiency / performance	Major problem	Best use or application
Multilevel Queue (MLQ)	Place processes into fixed queues based on type or priority	Usually	Efficient separation of interactive, batch, and system jobs	Processes cannot normally move between queues; lower queues may starve	Systems with clearly defined process classes
Multilevel Feedback Queue (MLFQ)	Move processes among queues according to observed behavior	Yes	Excellent interactive response without knowing burst lengths	Complex to tune; processes may manipulate the scheduler	General-purpose operating systems
Lottery Scheduling	Give processes tickets; randomly select a winning ticket	Yes	Simple probabilistic proportional sharing; low selection cost with suitable structures	Short-term results can be unpredictable; weak real-time guarantees	Research systems and proportional resource allocation
Stride Scheduling	Deterministic version of lottery scheduling using virtual strides	Yes	Predictable proportional sharing	Requires careful state management; new-process handling can be complex	Systems requiring deterministic proportional allocation
Fair-Share Scheduling	Allocate CPU according to users or groups rather than individual processes	Yes	Prevents one user with many processes from taking most of the CPU	Group weights and policies can be difficult to configure	Multiuser servers and HPC environments
Completely Fair Scheduler (CFS)	Run the process with the smallest virtual runtime	Yes	Good general-purpose fairness; efficient selection using a balanced tree	Not optimal for strict deadlines or every workload	Linux general-purpose process scheduling
Proportional Share	Allocate CPU according to assigned weights	Yes	Flexible and fair when weights reflect policy	Incorrect weights produce unfair performance	Containers, virtual machines, and cloud systems
Guaranteed Scheduling	Track the CPU share each process should have received and favor processes below that share	Yes	Strong long-term fairness	Accounting and tracking introduce overhead	Multiuser and resource-controlled systems

Real-Time Scheduling Algorithms

Scheduling method	Main idea	Priority type	Efficiency / performance	Major problem	Best use or application
Earliest Deadline First (EDF)	Run the ready task with the nearest deadline	Dynamic	High processor utilization; optimal on one CPU under common assumptions	Overload can cause many deadlines to fail unpredictably	Dynamic real-time systems
Rate Monotonic Scheduling (RMS)	Give higher priority to tasks with shorter periods	Fixed	Predictable and easy to analyze	Lower theoretical CPU utilization than EDF	Periodic hard real-time systems
Deadline Monotonic (DM)	Give higher priority to tasks with shorter relative deadlines	Fixed	Effective when task deadlines differ from their periods	Requires known task parameters; less flexible than EDF	Fixed-priority real-time systems
Least Laxity First (LLF)	Run the task with the least scheduling flexibility: deadline – remaining time	Dynamic	Reacts quickly to tasks close to missing deadlines	Priorities change frequently, causing many preemptions	Specialized real-time systems
Round Robin with Priorities	Schedule priority classes first, then use RR within each class	Fixed or dynamic	Combines responsiveness with priority control	Low-priority tasks may starve	Soft real-time and interactive systems

Overall Comparison

Algorithm	Response time	Average waiting time	Fairness	Starvation risk	Scheduling overhead
FCFS	Poor	Poor	Moderate	None	Very low
SJF/SPN	Moderate	Excellent	Poor	High	Low–moderate
SRTF	Excellent for short jobs	Excellent	Poor	High	High
Round Robin	Good	Moderate	Good	Very low	Moderate–high
Priority	Depends on priority	Variable	Poor without aging	High	Low–moderate
HRRN	Moderate	Good	Good	Low	Moderate
MLQ	Good for upper queues	Variable	Poor across queues	High	Moderate
MLFQ	Excellent for interactive jobs	Good	Good when tuned	Low–moderate	High
Lottery	Good over time	Variable	Probabilistically fair	Possible	Moderate
Stride	Good	Good	Proportionally fair	Low	Moderate
CFS	Good	Good	Strong general fairness	Very low	Moderate
EDF	Deadline-oriented	Not the primary goal	Deadline-based	Possible under overload	High
RMS	Predictable	Not the primary goal	Priority-based	Possible	Low–moderate

Chapter 2 Processes and Threads -- Summary

PROCESSES

The Process Model
Process Creation & Termination
Process Hierarchies
Process States
Implementation of Processes
Modeling Multiprogramming

THREADS

The Classical Thread Model
POSIX Threads
User / Kernel Threads

EVENT-DRIVEN SERVERS

SYNCHRONIZATION AND IPC

Race Conditions, Critical Regions, Mutual Exclusion with
Busy Waiting, Sleep and Wakeup
Semaphores, Mutexes, Monitors, Message Passing
9 Barriers
Priority Inversion
Read-Copy-Update

SCHEDULING

Batch Systems
Interactive Systems
Real-Time Systems
Policy Versus Mechanism
Thread Scheduling

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.